

Robotics French Cup Strategy Simulator

Agent Author's Guide

Contents

1	Overview	2
2	The game, conceptually	2
3	Game rules in detail	2
4	Getting started	3
4.1	Running a game	3
4.2	Choosing which agent plays	3
5	Writing your own agent	3
5.1	Agent memory	3
6	The agent interface (interface)	5
6.1	Information functions	5
6.2	Action functions	5
7	Data classes	6
8	Constants reference	6

1 Overview

A 2-player game simulating the 2026 Robotics French Cup, to be played by python agents. Two circular robots compete on a rectangular map to collect, recolor, and deposit “boxes” for points.

As an agent author, you will:

- write Python modules in `agents/` that decide what your robot does each tick,
- select which agent plays which player in `agents_config.py`,
- run and watch games with the `play.py` launcher.

2 The game, conceptually

The map

A rectangle containing non-overlapping rectangular areas: player 0’s area, player 1’s area, and one or more *scoring areas*.

Boxes

Identical-sized rectangles scattered on the map. Each box has a **color** (0 or 1) and can be picked up by robots.

Robots

Each player controls one circular robot starting in their own area. A robot can rotate freely, move forward/backward, pick up a nearby box, change the color of a nearby box, or lay down a box it is holding.

Turns

Time is discretized into *ticks*. Each tick, player 0 then player 1 gets to call `make_decision()` once. **Exactly one action** may be taken per turn.

Scoring

Computed at game end:

- every box sitting in player X ’s own area $\rightarrow +\text{config.POINTS_OWN_AREA}$ for X (regardless of the box’s color);
- every box sitting in a scoring area whose color is $X \rightarrow +\text{config.POINTS_SCORING_AREA}$ for X ;
- boxes currently held by a robot score nothing.

3 Game rules in detail

- **One action per turn:** `make_decision()` may call exactly one action function. `rotate` doesn’t count as an action and can be called an arbitrary number of times every turn.
- **Lay-down placement:** fixed distance (`config.LAY_DOWN_DISTANCE`) directly in front of the robot, oriented along the robot’s current facing direction. Fails if the spot is out of map bounds or overlaps another box.
- **Box rotation:** boxes take the robot’s orientation when laid down.
- **Initial layout:** fully config-defined and fixed.
- **Failed actions don’t consume the turn:** if an action raises `GameError` (e.g. `lay_down` blocked, `pickup` out of reach), your turn is *not* used up, so you can try something else.
- **Movement collision:** the robot stops just before hitting the map edge, the other robot, or a laid-down box.

- **Held boxes don't score** and are not physical obstacles for movement.

4 Getting started

4.1 Running a game

Use `play.py` with your regular system Python (`python` / `python3` / `py`) from the project root.

```
python play.py run                # run a game, print progress +
    final score
python play.py run --quiet        # suppress per-tick error
    logging
python play.py run --save         # also save a timestamped game
    to saved_games/
python play.py run --save --view  # run, save, then immediately
    replay it
python play.py view <history.json> # replay a game from saved_games
    / (or any path)
```

4.2 Choosing which agent plays

Edit `agents_config.py` at the project root:

```
PLAYER0_AGENT = "agents.simple_agent"
PLAYER1_AGENT = "agents.idle_agent"
```

Each string is a module path under `agents/`, loaded dynamically. Both slots can name the *same* module to make an agent play against itself.

5 Writing your own agent

Create `agents/my_agent.py` defining a class `Agent` with a `make_decision(self)` method:

```
import interface

class Agent:
    def make_decision(self):
        player = interface.me()
        accessible = interface.get_accessible_boxes(player)
        if accessible:
            interface.pickup(accessible[0].id)
        else:
            interface.move(10.0)
```

Then point `agents_config.PLAYER0_AGENT` (or `PLAYER1_AGENT`) at `"agents.my_agent"` and run `python play.py run -view`.

Look at `agents/random_agent.py` for an example.

5.1 Agent memory

The runner creates one `Agent()` *instance* per player, even when `PLAYER0_AGENT` and `PLAYER1_AGENT` both name the same module. That means your agent can have a memory if you need it:

```
class Agent:
    def __init__(self):
```

```
        self.visited = set()

    def make_decision(self):
        player = interface.me()
        self.visited.add(interface.get_position(player))
        # ...
```

6 The agent interface (interface)

Agents import `interface` (and typically `config` for constants like `config.MAX_MOVE_SPEED`).

6.1 Information functions

Free to call, any number of times.

Function	Returns
<code>interface.me()</code>	This agent's player index (0 or 1)
<code>interface.opponent()</code>	1 - <code>me()</code>
<code>interface.get_position(player)</code>	(x, y) center of player's robot
<code>interface.get_orientation(player)</code>	Unit vector player's robot is facing
<code>interface.get_map()</code>	The <code>Map</code> object (<code>.width</code> , <code>.height</code> , <code>.areas</code>)
<code>interface.get_area_containing(position)</code>	The <code>Area</code> that contains <code>position</code> , or <code>None</code> if it is in no area
<code>interface.get_area(area_id)</code>	The <code>Area</code> with that id, or <code>None</code>
<code>interface.get_area_center(area_id)</code>	(x, y) center of that area
<code>interface.get_boxes()</code>	All <code>Box</code> objects (on ground and held)
<code>interface.get_box(box_id)</code>	The <code>Box</code> with that id, or <code>None</code>
<code>interface.get_accessible_boxes(player)</code>	Boxes on the ground within reach of <code>player</code>
<code>interface.get_boxes_held(player)</code>	Boxes currently held by <code>player</code>
<code>interface.is_accessible(player, box_id)</code>	Is the box on the ground and within reach?
<code>interface.is_colliding(player, amount)</code>	Would moving forward by <code>amount</code> collide (edge/robot/box)?
<code>interface.get_box_distance(player, box_id)</code>	Circle-to-rectangle distance, robot → box
<code>interface.get_box_direction(player, box_id)</code>	Unit vector from robot center toward box center
<code>interface.get_area_distance(player, area_id)</code>	Distance from robot center to the area rectangle (0 if inside)
<code>interface.get_area_direction(player, area_id)</code>	Unit vector from robot center toward area center
<code>interface.get_score(player)</code>	Score <code>player</code> would get if the game ended right now

6.2 Action functions

Exactly one per `interface.make_decision()` (except `rotate`, which is free).

Function	Effect
<code>interface.move(amount)</code>	Move forward <code>amount</code> (negative = backward), capped at <code>config.MAX_MOVE_SPEED</code> ; stops just before any collision
<code>interface.rotate(new_direction)</code>	Face <code>new_direction</code>
<code>interface.pickup(box_id)</code>	Pick up an accessible box (fails if too far or hands full)
<code>interface.set_color(box_id, color)</code>	Recolor an accessible box to 0 or 1

Function	Effect
<code>interface.lay_down(box_id)</code>	Place a held box in front of the robot (fails if blocked / out of bounds)

All action functions may raise:

- `interface.ActionError` if a second action is attempted in the same turn.
- `game.GameError` if the action itself is invalid (out of reach, blocked, etc.). This does **not** consume the turn, so you can recover and try something else.

7 Data classes

These are the objects returned by `interface` functions.

Box(id, position, orientation, color, owner)

A box on the map. `owner == -1` means it is sitting on the ground; otherwise `owner` is the index of the player currently holding it. `color` is 0 or 1; `orientation` is a unit vector along its width axis.

Area(id, type, x, y, width, height)

An axis-aligned rectangular zone. `type` is 0 (player 0's area), 1 (player 1's area), or 2 (a scoring area).

Map(width, height, areas)

The static map: overall dimensions plus the list of all **Area** objects (player areas and scoring areas together).

8 Constants reference

Map

Constant	Value	Meaning
<code>config.MAP_WIDTH</code>	1000.0	Width of the playing field, in game units (the x extent; recall (0, 0) is the top-left corner).
<code>config.MAP_HEIGHT</code>	600.0	Height of the playing field (the y extent).

Areas

Constant	Value	Meaning
<code>config.PLAYER0_AREA</code>	(0, 200, 150, 200)	(x, y, width, height) rectangle defining player 0's home area.
<code>config.PLAYER1_AREA</code>	(850, 200, 150, 200)	Same, for player 1 (<code>Area.type == 1</code>).
<code>config.SCORING_AREAS</code>	list of 2 rects	List of (x, y, width, height) rectangles, each becoming an Area with <code>type == 2</code> .

Robot starting positions and orientations

Constant	Value	Meaning
<code>config.PLAYER0_START</code>	(75, 300)	(<i>x</i> , <i>y</i>) spawn position of player 0’s robot — inside <code>PLAYER0_AREA</code> .
<code>config.PLAYER0_START_ANGLE</code>	0.0	Initial facing angle of player 0’s robot, in degrees measured from the $+x$ axis (0° = facing right, toward the center of the map).
<code>config.PLAYER1_START</code>	(925, 300)	Spawn position of player 1’s robot.
<code>config.PLAYER1_START_ANGLE</code>	80.0	Initial facing angle of player 1’s

Robot physics

Constant	Value	Meaning
<code>config.ROBOT_RADIUS</code>	20.0	Radius of the circular robot body.
<code>config.MAX_MOVE_SPEED</code>	10.0	The largest <code> amount </code> a single <code>move</code> call will actually apply, per tick.
<code>config.MAX_BOXES_HELD</code>	3	Maximum number of boxes a robot can carry at once.

Box dimensions

Constant	Value	Meaning
<code>config.BOX_WIDTH</code>	40.0	Width of a box.
<code>config.BOX_HEIGHT</code>	25.0	Height of a box.

Interaction distances

Constant	Value	Meaning
<code>config.ACCESSIBILITY_RADIUS</code>	25.0	Maximum circle-to-rectangle distance (robot center $\rightarrow$ nearest point on the box) at which a box counts as “accessible” (i.e. within reach for <code>pickup</code> and <code>set_color</code>).
<code>config.LAY_DOWN_DISTANCE</code>	45.0	Fixed distance from the robot’s center to the <i>center</i> of a box placed via <code>lay_down</code> .

Scoring

Constant	Value	Meaning
<code>config.POINTS_OWN_AREA</code>	3	Points awarded to player <i>X</i> for each box sitting in <i>X</i> ’s own area.
<code>config.POINTS_SCORING_AREA</code>	5	Points awarded to player <i>X</i> for each box sitting in a scoring area whose color matches X .

Timing

Constant	Value	Meaning
<code>config.DT</code>	0.1	Seconds represented by one tick.
<code>config.GAME_DURATION</code>	120.0	Total game length, in seconds ($= DT \times TOTAL_TICKS$).
<code>config.TOTAL_TICKS</code>	1200	Number of ticks in a game.

Initial boxes

Constant	Value	Meaning
<code>config.INITIAL_BOXES</code>	list of 11 tuples	The fixed starting layout: each entry is <code>(x, y, angle_degrees, color)</code> — the box's initial center position, its orientation expressed as an angle in degrees from the $+x$ axis, and its starting color (0 or 1).